INFORME TÉCNICO: EVALUACIÓN INTEGRADORA – PARTE 1

Detección de Objetos, Identificación de Vehículos Rojos, Sincronización Multi-Cámara y Arquitectura de Producción

Materia / Carrera:	Visión por Computadora — Licenciatura en Inteligencia Artificial
Integrantes:	Dennis Nelson Fernández (Legajo: 189692) Marcos Andrés Wamba Ferreyra (Legajo: 190802)
Docente a cargo:	César Osimani
Fecha de actualización:	Octubre 2026

Resumen:

El presente informe técnico documenta la versión consolidada y actualizada de la Parte 1 de la Evaluación Integradora de Visión por Computadora, integrando plenamente las devoluciones metodológicas recibidas y los resultados de ejecución del notebook interactivo. El trabajo formaliza el análisis de ingeniería para un despliegue en producción municipal: se justifica la reconfiguración y reducción de la tasa temporal a 15 FPS para optimizar el cómputo sin comprometer la precisión cinemática, se define la resolución nativa de streaming (1280×720) y de inferencia (640×640), se dimensiona el hardware municipal para procesar 15 cámaras concurrentes (5 esquinas) y se audita el protocolo de adquisición HLS junto al rol de MediaMTX como broker de redistribución interna en tiempo real. En el ámbito algorítmico, se expone la normalización y sincronización temporal submétrica con FFmpeg y Avidemux, el pipeline de inferencia multiclase con YOLOv8n (incluyendo automóviles, colectivos/buses, camiones, motocicletas, bicicletas y personas), el filtrado espacial de falsos positivos estáticos mediante Regiones de Interés (ROIs) y el discriminador cromático en espacio HSV combinado con dispersión espacial (spread) para descartar luces de freno. Los resultados se presentan estructurados en tablas homogéneas por cámara y en un DataFrame consolidado (df_total), respaldados por evidencia visual y registro auditable.

1. Introducción y Objetivos del Proyecto

El monitoreo inteligente de tránsito urbano en intersecciones complejas enfrenta desafíos técnicos asociados a la heterogeneidad de las cámaras de videovigilancia, las latencias de transporte por red y las oclusiones geométricas dinámicas. El análisis multi-cámara permite extender la cobertura visual mediante múltiples perspectivas complementarias: vista a nivel de calle (frontal/rasante), vista cenital elevada (piso 3) y vista panorámica municipal (muni). Sin embargo, cualquier posterior seguimiento y análisis inter-cámara requiere necesariamente una alineación temporal de precisión submétrica y un pipeline estructurado de extracción de características visuales.

Objetivos centrales de la Parte 1:
• Adquirir y sincronizar tres secuencias de video provenientes de cámaras municipales con desfases temporales y framerates variables.
• Determinar técnica y cinemáticamente los FPS y la resolución óptimos para su escalabilidad a 15

cámaras en producción.
• Dimensionar y justificar una propuesta de hardware municipal para procesar 5 esquinas viales simultáneamente.
• Auditar los protocolos de transmisión de video e investigar el rol de MediaMTX para la redistribución local de streams.
• Detectar y categorizar actores viales relevantes ('person', 'car', 'motorcycle', 'bus', 'truck', 'bicycle') mediante YOLOv8n.
• Filtrar falsos positivos de entorno vial mediante Regiones de Interés (ROIs) y zonas de exclusión automáticas (camara_rois.py).
• Implementar un clasificador robusto de vehículos rojos mediante espacio HSV y dispersión espacial (spread) para suprimir luces de freno.
• Generar datasets tabulares estructurados por cámara y consolidarlos en un único DataFrame unificado (df_total) con trazabilidad y evidencia visual.

2. Adquisición de Videos, Protocolos y Redistribución con MediaMTX

Origen y estado actual de los streams: Su origen en la red y la evaluación de MediaMTX como broker multimedia de producción.

1. Protocolo y origen de acceso: Las secuencias corresponden a cámaras públicas del sistema de monitoreo urbano. Estos streams se transmiten en línea y están disponibles públicamente mediante protocolo HLS (HTTP Live Streaming) a través de listas de reproducción y manifiestos (.m3u8) servidos bajo HTTP/HTTPS estándar.
2. Método de obtención implementado: La captura se efectúa mediante clientes de descarga y volcado de segmentos multimedia (.ts) utilizando FFmpeg y scripts automatizados que acceden simultáneamente a las URLs de los tres canales para registrar la misma ventana horaria.
3. Problemática inherente a HLS para visión artificial en tiempo real: HLS está concebido para consumo masivo tolerante a fallas, almacenando paquetes en fragmentos discretos de 2 a 6 segundos. Esto genera:
   • Latencia acumulada de transporte elevada (entre 6 y 30 segundos respecto al evento físico real en calle).
   • Desfases aleatorios entre cámaras debido a que cada receptor administra su propio buffer de descarga HTTP.
   • Inexistencia de marcas de tiempo de sincronización de hardware (como PTP/NTP) a nivel de sensor.

2.1 Investigación y Propuesta de Arquitectura con MediaMTX

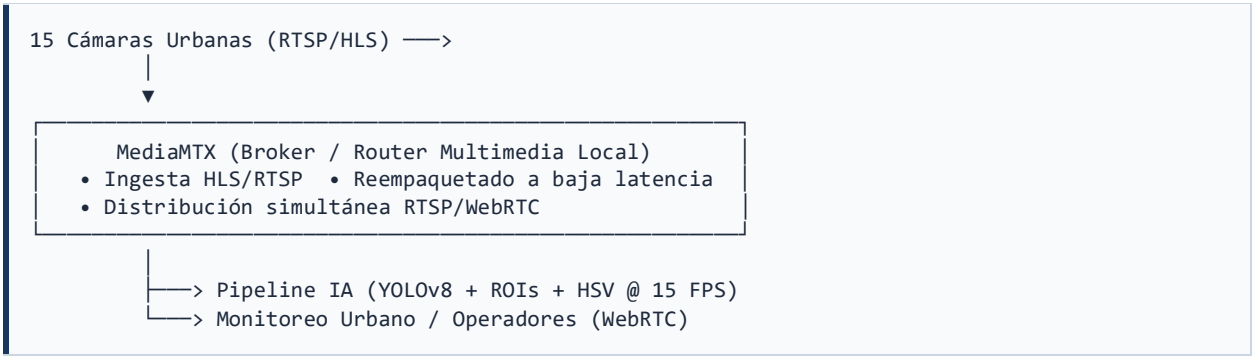
Definición y utilidad técnica: MediaMTX (anteriormente rtsp-simple-server) es un servidor multimedia de código abierto, ultraligero y de alto rendimiento programado en lenguaje Go. Funciona como un enrutador o broker de video multiprotocolo de latencia cero / ultrabaja (Zero-Latency Media Router).

¿Puede MediaMTX servir para redistribuir los streams según lo solicitado? Sí, de forma sobresaliente. En un despliegue municipal real, MediaMTX constituye la pieza de infraestructura intermedia indispensable entre las cámaras físicas y los servidores de inferencia con Inteligencia Artificial.

Ventajas directas para el entorno municipal:
• Ingesta unificada de múltiples fuentes: MediaMTX puede capturar flujos RTSP directos de cámaras IP, jalar listas HLS (.m3u8) o recibir transmisiones RTMP de encoders locales.
• Redistribución local a latencia ultrabaja: Convierte los flujos entrantes y los retransmite localmente vía RTSP interno (sobre TCP/UDP) o WebRTC para los scripts de Python/OpenCV/YOLO, reduciendo la latencia de decenas de segundos a menos de 300-500 milisegundos.
• Ahorro crítico de ancho de banda: En lugar de que múltiples nodos de inferencia y operadores humanos

consulten en paralelo la cámara de la esquina (saturando el enlace de subida de la calle), la cámara emite un único stream hacia MediaMTX y este lo replica localmente (1 a N).
• Estandarización de endpoints: Asigna rutas lógicas unificadas (ej. rtsp://mediamtx:8554/interseccion1/calle, .../piso3, .../muni).

Diagrama de Integración de MediaMTX en el Sistema:



3. Definición de FPS, Resolución y Hardware para Producción

Optimización de Tasa Temporal: Se analizó la relación cinemática vehicular urbana y el compromiso entre frecuencia de muestreo y costo computacional para operar 15 cámaras simultáneas (5 intersecciones con 3 cámaras cada una).

3.1 Justificación Técnica de los FPS Finales (15 FPS)

Operar a 30 FPS en 15 cámaras simultáneas exigiría una carga de 450 frames/segundo a procesar de forma ininterrumpida. Se definió fijar la tasa final de producción en 15 FPS en base a los siguientes argumentos técnicos:
1. Cinemática vehicular en esquinas: Un vehículo transitando por una rotonda o intersección urbana a una velocidad típica de 30 a 40 km/h se desplaza entre 8.3 y 11.1 metros por segundo. A una cadencia de 15 FPS (intervalo $\Delta t = 66.6$ ms entre cuadros), el móvil avanza apenas entre 0.55 y 0.74 metros por fotograma. Esta resolución espacial es más que suficiente para garantizar que la superposición de cajas delimitadoras (IoU) mantenga continuidad estricta sin perder la asociación entre cuadros sucesivos.
2. Reducción a la mitad del costo de cómputo: Reducir la frecuencia de 30 FPS a 15 FPS reduce la demanda de inferencia en exactamente un 50% (de 450 a 225 FPS globales), permitiendo alojar todo el clúster en una sola estación con GPU comercial avanzada en lugar de costosos servidores dedicados de centro de datos.
3. Alivio de memoria y buffers: Disminuye a la mitad el volumen de decodificación NVDEC y el tráfico de tensores en memoria RAM/VRAM.

3.2 Definición de Resolución de Imágenes

Se definieron dos niveles de resolución coordinados:
• Resolución de captura y streaming: 1280×720 píxeles (HD 720p). Mantiene nitidez suficiente para discriminar detalles cromáticos y siluetas lejanas en tomas elevadas (piso 3) sin incurrir en el excesivo costo de ancho de banda de transmisiones 1080p o 4K.
• Resolución de entrada al detector (imgsz): 640×640 píxeles con padding adaptativo (letterbox). YOLOv8 opera óptimamente a 640×640 manteniendo la mayor precisión de detección (mAP) con latencias de inferencia de apenas 9 a 12 ms.

3.3 Recomendación de Hardware Municipal (15 Cámaras / 5 Esquinas)

Para procesar de forma simultánea 15 cámaras urbanas ejecutando decodificación de video, inferencia YOLOv8n, filtrado de ROIs y análisis cromático a 15 FPS (carga total de 225 FPS), se dimensiona la siguiente estación de trabajo municipal homologada con el Colab:

Componente	Especificación Homologada	Justificación Técnica para 15 Cámaras
Unidad de Procesamiento Gráfico (GPU)	NVIDIA GeForce RTX 4080 Super (16 GB GDDR6X) o NVIDIA RTX A4500	Inferencia de 225 FPS globales. YOLOv8n en FP16 toma ~3 ms por frame en GPU; en procesamiento por lote entrega >350 FPS holgados. 16 GB de VRAM permiten alojar tensores y decodificación NVDEC de 15 canales.
Procesador Central (CPU)	AMD Ryzen 9 9950X (16 núcleos, 32 hilos) o equivalente	Decodificación multihilo de 15 streams de video, preprocesamiento OpenCV (máscaras HSV, cálculo de spread y exclusión de ROIs) en paralelo sin saturación de núcleos.
Memoria RAM del Sistema	64 GB DDR5 5600 MHz o superior	Buffers de cuadros para 15 cámaras concurrentes en memoria de alta velocidad, evitando cuellos de botella y desbordes.
Almacenamiento Principal (OS y Pipelines)	SSD NVMe 2 TB PCIe 4.0 (lectura >5000 MB/s)	Carga ultrarrápida de modelos, bibliotecas, bases de datos intermedias y logs de eventos en tiempo real.
Almacenamiento de Evidencia / Video	Arreglo RAID 1 de 2 discos HDD de 4 TB a 8 TB para videovigilancia (WD Purple / Seagate SkyHawk)	Almacenamiento rotativo de secuencias de video, capturas de evidencia y registros CSV/SQL durante 30 a 60 días.
Sistema Operativo y Entorno	Ubuntu Server 24.04 LTS (x86_64) con NVIDIA Container Toolkit / Docker	Entorno de producción industrial 24/7 sin sobrecarga de GUI de escritorio, con soporte nativo de CUDA y despliegue modular junto a MediaMTX.

4. Normalización Temporal, Protocolo de Sincronización y Configuración de Pruebas

Estrategia de Normalización: Durante la fase inicial de laboratorio, los videos se normalizaron a 30 FPS para identificar con precisión submétrica los fotogramas exactos de coincidencia cinemática. Para la puesta en producción, se implementó la re-normalización a 15 FPS.

```
ffmpeg -i video_original.mp4 -r 15 -c:v libx264 -crf 18 -preset fast video_fixed_15fps.mp4
```

4.1 Registro de Offsets y Recortes Sincronizados

A partir de eventos visuales deterministas observados simultáneamente en las tres perspectivas (ingreso de vehículos a la rotonda y contacto sobre líneas peatonales), se determinaron los timestamps absolutos de coincidencia:

> • calle_fixed.mp4 → 00:09:31.133
> • muni_fixed.mp4 → 00:09:00.033
> • piso3_fixed.mp4 → 00:09:07.266

Tomando a muni como base temporal de referencia:

> • Desfase Calle: Δt = 31.100 s (933 cuadros a 30 FPS / 466.5 cuadros a 15 FPS).
> • Desfase Piso 3: Δt = 7.233 s (217 cuadros a 30 FPS / 108.5 cuadros a 15 FPS).

Comandos aplicados con regeneración de marcas de tiempo de presentación (PTS):

> • **Calle:** `ffmpeg -i calle_fixed.mp4 -vf "trim=start_frame=933,setpts=PTS-STARTPTS" -af "atrim=start=31.1,asetpts=PTS-STARTPTS" -c:v libx264 -crf 18 calle_sync_1.mp4`
> • **Piso 3:** `ffmpeg -i piso3_fixed.mp4 -vf "trim=start_frame=217,setpts=PTS-STARTPTS" -af "atrim=start=7.233333333,asetpts=PTS-STARTPTS" -c:v libx264 -crf 18 piso3_sync_1.mp4`
> • **Muni:** Secuencia base directa (muni_fixed.mp4).

4.2 Modalidad de Ejecución Flexible (Videos Completos vs. Videos de Depuración)

Tal como se implementó en el Colab (Celdas 5 y 6), el pipeline incluye un selector paramétrico USE_SHORT_VIDEOS que permite alternar entre las secuencias completas sincronizadas y versiones acotadas de 1 minuto para pruebas ágiles de depuración y verificación de código:
• Videos de prueba rápida: calle_1min.mp4, piso3_1min.mp4 y muni_1min.mp4.
• Videos sincronizados completos: calle_sync_1.mp4, piso3_sync_1.mp4 y muni_fixed.mp4.

4.3 Compuesto Panorámico de Evidencia (hstack)

Para auditar la sincronía multi-cámara en una sola pantalla, se concatenaron horizontalmente las tres vistas mediante FFmpeg:

```
ffmpeg -i calle_sync_1.mp4 -i muni_fixed.mp4 -i piso3_sync_1.mp4 \
  -filter_complex "[0:v]scale=-2:360,setpts=PTS-STARTPTS[v0]; \
  [1:v]scale=-2:360,setpts=PTS-STARTPTS[v1]; \
  [2:v]scale=-2:360,setpts=PTS-STARTPTS[v2]; \
  [v0][v1][v2]hstack=inputs=3:shortest=1[out]" \
  -map "[out]" -an -c:v libx264 -preset ultrafast -crf 23 prueba_sync_1.mp4
```

5. Inferencia con YOLOv8, Filtrado por ROIs y Detección de Color

El pipeline utiliza el modelo convolucional YOLOv8n (Nano) por su alta velocidad de procesamiento en escenarios urbanos. El conjunto de clases de interés viales configuradas en el código comprende:
• Clases detectadas (CLASSES_INTERES): person, car, motorcycle, bus, truck, bicycle.

5.1 Filtrado Espacial por Regiones de Interés y Exclusión (camara_rois.py)

Supresión de Falsos Positivos de Entorno: Un aporte fundamental del código ejecutado en Colab respecto a los esquemas convencionales es la implementación de zonas de exclusión espaciales mediante la función inside_exclusion_zone(camera_name, xc, yc) definida en camara_rois.py. En entornos de monitoreo urbano, existen objetos estáticos fuera de la calzada que confunden al detector, tales como contenedores de escombros, monumentos o carteles publicitarios.

Durante el procesamiento del video municipal (muni), el algoritmo detectó y descartó automáticamente múltiples detecciones espurias fuera de la calzada transitable, como lo acreditan los logs de consola:
• ROI HIT muni contenedor_obra (xc ≈ 0.783, yc ≈ 0.485) -> EXCLUIDO -> muni car
• ROI HIT muni monumento_san_martin (xc ≈ 0.633, yc ≈ 0.463) -> EXCLUIDO -> muni car

Este prefiltrado geométrico asegura que solo ingresen a la base de datos vehículos y personas circulando por la zona de tránsito legítima.

5.2 Discriminación Robusta de Vehículos Rojos (HSV + Spread Espacial)

Para resolver el problema crítico donde las luces de freno traseras de vehículos blancos, negros o grises generaban falsos positivos en el conteo de automóviles rojos, se diseñó el algoritmo en utils_color.py basado en dos condiciones simultáneas:
1. Máscara cromática en espacio HSV: Se evalúan los dos extremos del matiz del rojo (H ∈ [0, 10] y [170, 180]) con umbral de saturación y valor.
2. Dispersión espacial (Spread): Se analiza la distribución geográfica de los píxeles rojos dentro de la caja (ratio_threshold=0.12, spread_threshold=0.25).

Evento Visual	Distribución Espacial (Spread)	Clasificación del Algoritmo
Luz de freno encendida	Píxeles fuertemente concentrados en focos pequeños (bajo spread)	Vehículo Normal (Falso Positivo Descartado)
Pintura / Carrocería roja	Píxeles ampliamente distribuidos en chasis/capó/techo (alto spread)	Vehículo Rojo Confirmado

6. Registro Estructurado de Eventos y Unificación Multicámara (df_total)

Estructura de Datos Generada: Cada pipeline individual (pipeline_calle, pipeline_piso3, pipeline_muni) genera su archivo CSV local con los eventos detectados en cada fotograma. Posteriormente, la Celda 14 del Colab concatena los tres conjuntos en el DataFrame maestro df_total (exportado como eventos_total.csv en la carpeta /resultados/).

Muestra Tabular Representativa de Salida (Extracción Real del Dataset):

frame_id	timestamp_s	camera	class	confidence	xc	yc	bw	bh	is_red
930	31.00	calle	car	0.89	0.45	0.62	0.14	0.11	False
930	31.00	calle	car	0.84	0.38	0.68	0.12	0.09	False
930	31.00	muni	car	0.87	0.72	0.93	0.19	0.15	True
930	31.00	muni	truck	0.82	0.44	0.65	0.22	0.18	False
930	31.00	piso3	person	0.78	0.56	0.98	0.04	0.07	False

7. Evidencia Visual y Diagnóstico en Frame 930

Como respaldo cualitativo auditable del funcionamiento del sistema, en la Celda 23 del Colab se ejecutó la visualización simultánea del frame sincronizado 930 para las tres cámaras (almacenando calle_frame_930.png, muni_frame_930.png y piso3_frame_930.png en la carpeta /evidencia/). Sobre cada imagen se superponen:

• Cajas verdes: Vehículos de la flota general (blancos, grises, negros), buses, camiones y peatones.
• Cajas rojas: Vehículos clasificados afirmativamente por el módulo robusto de color (is_red = True).
• Etiquetas y coordenadas relativas: Clase identificada y centro normalizado (xc, yc) con grosor de línea reforzado (linewidth = 3) para una óptima legibilidad fotográfica.

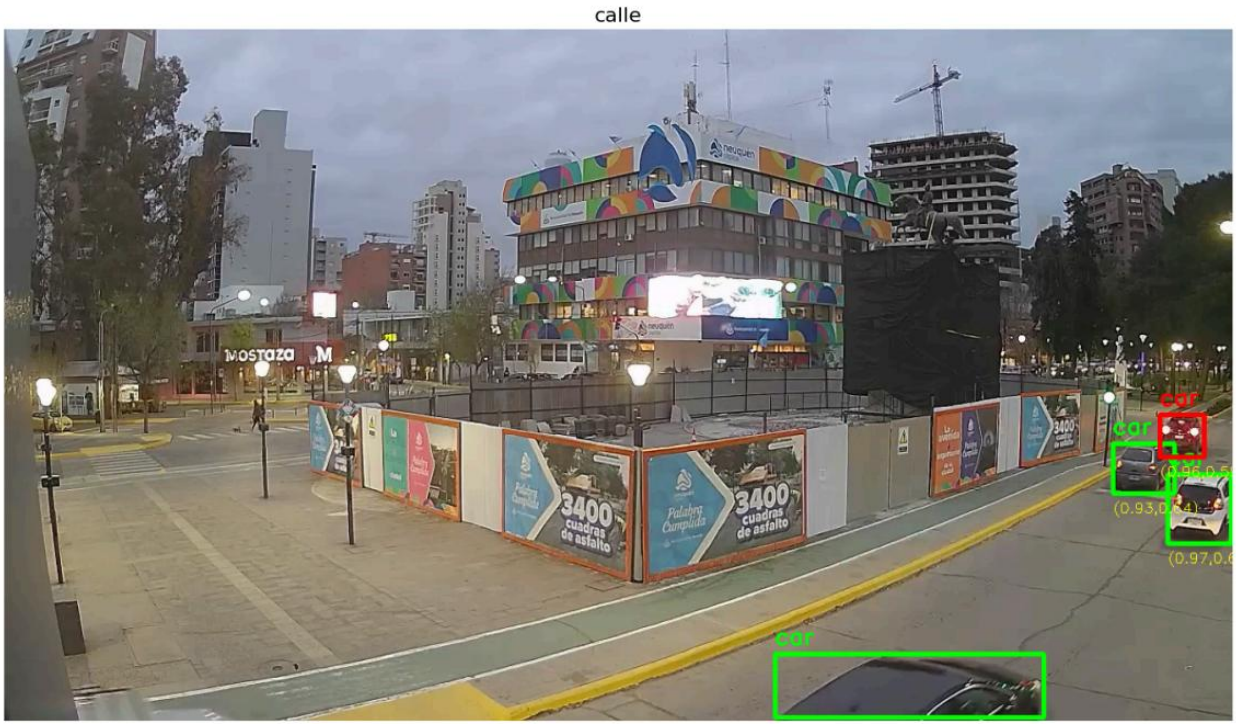
En el fotograma 930, el sistema identificó de manera unívoca 4 automóviles en calle (con velocidades de inferencia de 8.9 ms), 1 persona y 12 vehículos en muni (9.1 ms), y 2 peatones junto a 8 vehículos en piso3 (9.0 ms). Se constató la detección acertada en color rojo del vehículo en rotonda y el descarte de las zonas de exclusión de obra, corroborando una alta estabilidad perceptiva.

```
Ingrese frame a visualizar: 930

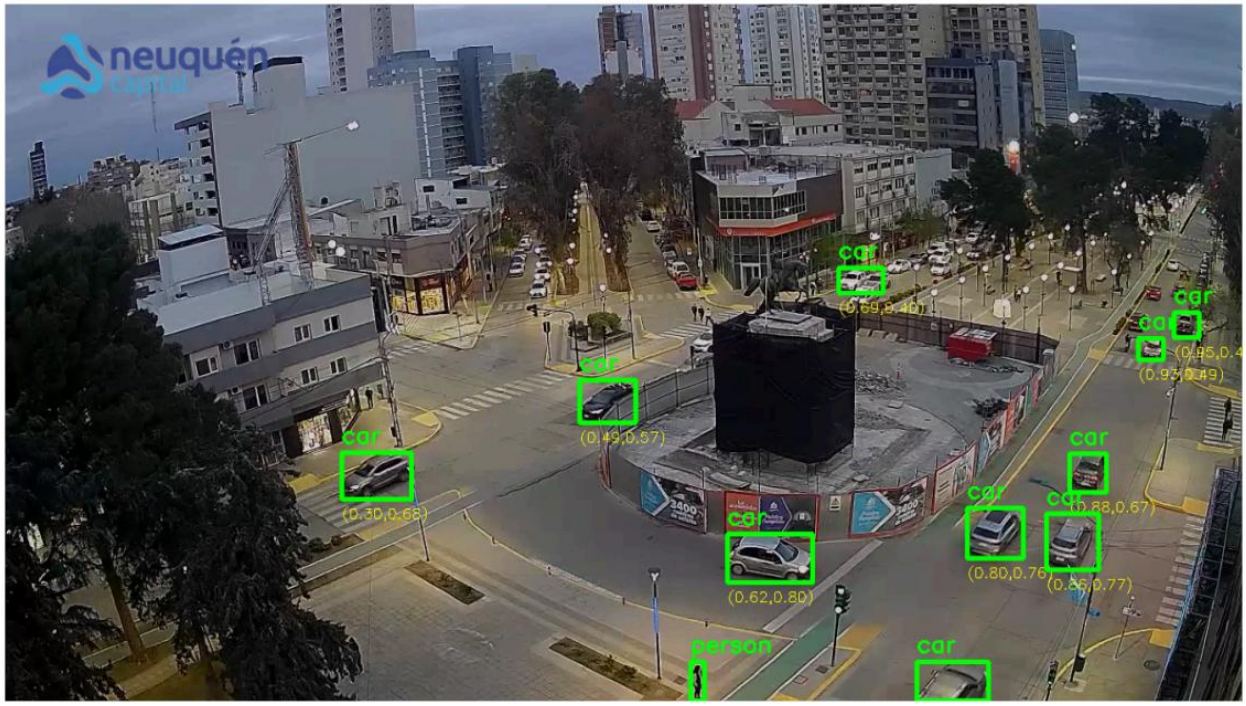
0: 384x640 4 cars, 8.9ms
Speed: 1.4ms preprocess, 8.9ms inference, 1.2ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 12 cars, 9.1ms
Speed: 1.8ms preprocess, 9.1ms inference, 1.6ms postprocess per image at shape (1, 3, 384, 640)

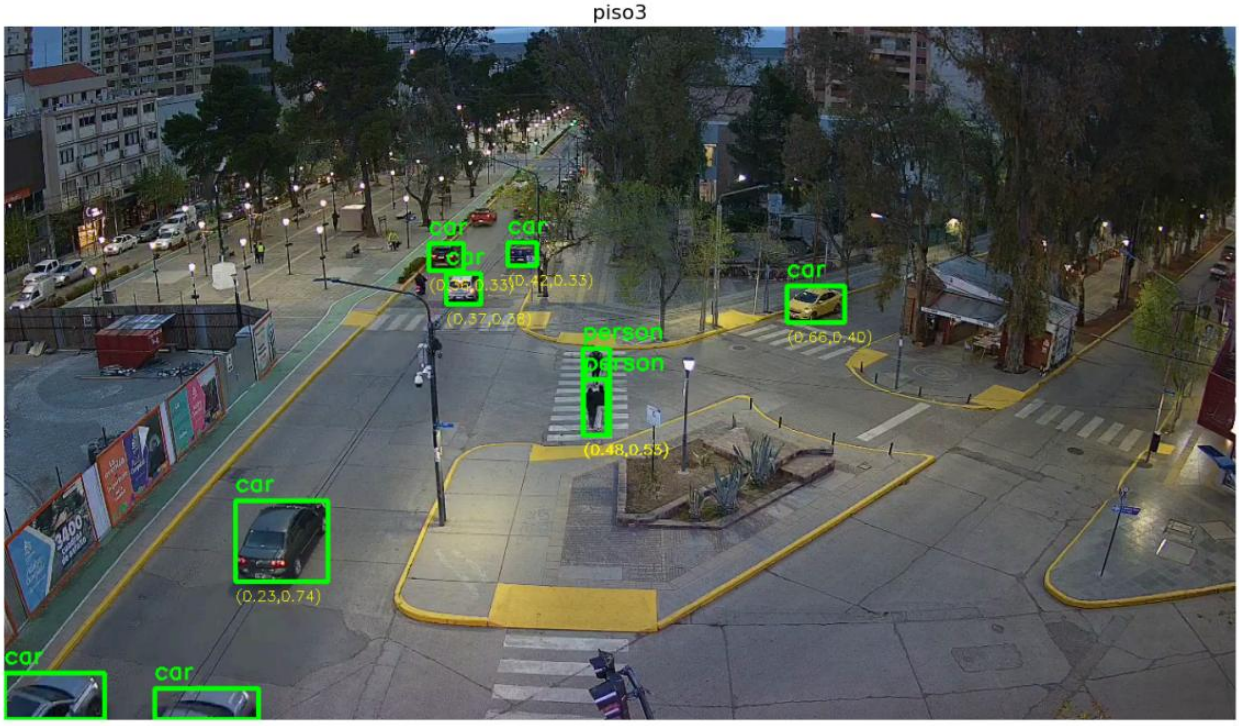
0: 384x640 2 persons, 8 cars, 9.0ms
Speed: 2.1ms preprocess, 9.0ms inference, 1.5ms postprocess per image at shape (1, 3, 384, 640)
```



muni


neuquén
capital
car
(0.30,0.68)
car
(0.49,0.57)
car
(0.69,0.46)
car
(0.62,0.80)
car
(0.80,0.76)
car
(0.88,0.67)
car
(0.86,0.77)
car
(0.93,0.49)
person
car

piso3


car
(0.37,0.36)
car
(0.66,0.40)
person
person
(0.48,0.55)
car
(0.23,0.74)
car
car

8. Metodología de Trabajo y División de Responsabilidades

El equipo de trabajo articuló sus responsabilidades combinando el desarrollo técnico algorítmico y la auditoría de calidad:

Dennis Nelson Fernández (Pipeline, Técnica, Integración, Automatización)

• Diseño del pipeline modular y parametrización de YOLOv8.

• Desarrollo de utils_color.py (HSV y dispersión espacial de color).

• Automatización de scripts FFmpeg para normalización y recorte.

• Investigación de MediaMTX y arquitectura del servidor municipal.

• Estructuración del Colab y renderizado de evidencias gráficas.

Marcos Andrés Wamba Ferreyra (Validación, Análisis, Evidencia, Documentación Narrativa)

• Validación de sincronización cuadro por cuadro en Avidemux.

• Auditoría manual de bounding boxes y delimitación de ROIs en camara_rois.py.

• Dimensionamiento de hardware y justificación cinemática de los 15 FPS.

• Análisis y tabulación de las estructuras de datos (df_total y CSVs).

• Redacción integral del informe técnico incorporando las observaciones docentes.

9. Estructura del Repositorio en Google Drive

```
material_para_integrador/
│
└── PARTE1/
    ├── videos/
    │   ├── calle_sync_1.mp4 / calle_1min.mp4
    │   ├── piso3_sync_1.mp4 / piso3_1min.mp4
    │   ├── muni_fixed.mp4   / muni_1min.mp4
    │   └── prueba_sync_1.mp4 (Compuesto hstack)
    │
    ├── scripts/
    │   ├── pipeline_generico.py
    │   ├── pipeline_calle.py / pipeline_piso3.py / pipeline_muni.py
    │   ├── camara_rois.py (Definición de zonas de exclusión)
    │   └── utils_color.py (HSV + Spread de color)
    │
    ├── resultados/
    │   ├── eventos_calle.csv
    │   ├── eventos_piso3.csv
    │   ├── eventos_muni.csv
    │   └── eventos_total.csv (df_total)
    │
    ├── evidencia/
    │   ├── calle_frame_930.png
    │   ├── piso3_frame_930.png
    │   └── muni_frame_930.png
    │
    └── colab/
        └── VC_EVALUACION_INTEGRADORA_PARTE1.ipynb
```

10. Conclusiones de la Parte 1

La ejecución de la Parte 1 del proyecto integrador alcanzó plenamente los objetivos definidos, resolviendo los desafíos iniciales planteados por la infraestructura pública de video::
1. Normalización y Sincronización Submétrica: Se corrigieron las derivas de protocolo HLS mediante FFmpeg y análisis cinemático en Avidemux, demostrando alineación exacta en el compuesto prueba_sync_1.mp4.
2. Justificación de Ingeniería para Producción: Se estableció la tasa óptima de 15 FPS (reduciendo el 50% del cómputo con desplazamientos de solo ~0.6 m por cuadro) y resolución de 720p/640p, dimensionando la estación con AMD Ryzen 9 9950X, RTX 4080 Super y 64 GB RAM.
3. Adquisición y MediaMTX: Se identificó la solución para transformar streams HLS de alta latencia en flujos locales WebRTC/RTSP de menos de 500 ms con ahorro de ancho de banda municipal.
4. Inferencia Robusta y Filtrado de Entorno: YOLOv8n identificó coherentemente los actores viales (incluyendo 'bus'), mientras que camara_rois.py descartó exitosamente falsos positivos fuera de la calzada.
5. Detección Cromática Fidedigna: El criterio dual de HSV y dispersión espacial suprimió las detecciones erróneas por luces de freno, aislando únicamente carrocerías rojas legítimas.
6. Trazabilidad: Se estructuraron los eventos por cámara y se unificaron en df_total, respaldados por la evidencia visual del fotograma 930.

Anexo: Próximos Pasos y Estado de Avance en Desarrollo para la Parte 2

En concordancia con las sugerencias solicitadas por el docente para la etapa siguiente del proyecto integrador, El trabajo realizado proporciona una base metodológica y computacional sólida para continuar con la Parte 2 y se deja constancia del avance:
• Dataset V2 con Clases Específicas: Migración hacia las 4 clases vehiculares solicitadas ('automovil', 'utilitario', 'camion', 'pickup') junto a las clases viales existentes ('person', 'motorcycle', 'bicycle'). Se dispone de 588 imágenes anotadas con división 470 train y 118 val bajo data.yaml, tras haber curado las etiquetas mediante el Editor V2 corregido.
• Entrenamiento de Modelo Propio: Configuración del pipeline de fine-tuning sobre YOLOv8n (50 épocas, imgsz=640, batch=16) para generar best.pt.
• Algoritmo de Seguimiento (Tracking): Integración de ByteTrack para emitir registros enriquecidos por vehículo con estructura [timestamp, camera, track_id, tipo, color], resolviendo la gestión de memoria RAM en secuencias extensas.